

Tier 1: Full-Feature Elastic-Net Nomogram at the Hour-6 Landmark

Contents

Overview	1
Load Libraries	1
Locate Data and Set Output Directories	1
Load Landmark Data	2
Feature Design	2
Outer 5-Fold CV with Inner Lambda Selection	2
Reconstruct Baseline Models	3
Compute Metrics	3
Final Model Coefficients	4
Export Out-of-Sample Predictions	4
Calibration Curve	4
Nomogram	5
Done	5

Overview

This script builds the full-feature penalized-logistic nomogram at the hour-6 landmark. It establishes the purely statistical upper bound for predicting incident Sepsis-3 within 12 hours of the hour-6 landmark by incorporating heavily-missing labs via an informative-missingness design (median-impute plus “was it measured” indicators). Evaluation uses honest, leakage-safe nested cross-validation with DeLong testing against the earlier baseline model.

Load Libraries

We load the required packages for elastic-net regression, ROC analysis, precision-recall curves, and regression modelling/nomograms.

```
suppressPackageStartupMessages({  
  library(glmnet); library(pROC); library(PRRROC); library(rms)  
})  
set.seed(486649)
```

Locate Data and Set Output Directories

This helper function searches candidate paths for data files, and then sets the output and figure directories.

```
find_data <- function(f) {  
  cands <- c(file.path(".././processed_data", f),  
             file.path("processed_data", f), f)  
  hit <- cands[file.exists(cands)][1]  
  if (is.na(hit)) stop("cannot find ", f)  
  hit  
}  
OUT <- dirname(find_data("landmark_h6.csv"))  
FIG <- if (dir.exists("figure")) "figure" else { dir.create("figure"); "figure" }
```

Load Landmark Data

We read in the hour-6 landmark dataset and print a summary of its size and onset rate.

```
lm6 <- read.csv(find_data("landmark_h6.csv"))
cat(sprintf("Loaded landmark_h6: %d stays, onset-within-12h rate %.3f\n",
           nrow(lm6), mean(lm6$onset_within_h)))
```

```
## Loaded landmark_h6: 48829 stays, onset-within-12h rate 0.068
```

Feature Design

The feature matrix combines vitals (near-complete after forward-fill), severity scores (SOFA, qSOFA, SIRS), labs with informative-missingness indicators, and 6-hour trajectory slopes. Labs and slopes use median/zero imputation with a binary “measured” flag to preserve the informative nature of missingness.

```
# Vitals (near-complete after forward-fill at h6): use directly.
vitals <- c("hr","resp_rate","map","spo2","temp_c","gcs")
# Scores allowed now (nomogram is a full-feature model, not the non-circular one).
scores <- c("sofa_total","qsofa","sirs")
# Labs: 70-88% missing at h6 and the missingness is INFORMATIVE (a drawn lactate
# marks a sicker/suspected patient). Median-impute + a "_msg" measured indicator.
labs <- c("lactate","creatinine","wbc","pf_ratio")
# 6h trajectory slopes: ~30% missing (fewer than 2 measurements). Impute 0
# (no trend) + a "_msg" indicator.
slopes <- c("hr_slope6","resp_rate_slope6","map_slope6","spo2_slope6")

df <- data.frame(row.names = seq_len(nrow(lm6)))
for (v in c(vitals, scores)) { # simple median-impute
  x <- lm6[[v]]; x[is.na(x)] <- median(x, na.rm = TRUE); df[[v]] <- x
}
for (v in labs) { # impute + informative flag
  x <- lm6[[v]]; df[[paste0(v, "_msg")]] <- as.integer(is.na(x))
  x[is.na(x)] <- median(x, na.rm = TRUE); df[[v]] <- x
}
for (v in slopes) { # impute 0 + flag
  x <- lm6[[v]]; df[[paste0(v, "_msg")]] <- as.integer(is.na(x))
  x[is.na(x)] <- 0; df[[v]] <- x
}
df$age <- lm6$age
df$female <- as.integer(lm6$gender == "F")
y <- lm6$onset_within_h
X <- as.matrix(df)
cat(sprintf("Design matrix: %d features\n", ncol(X)))
```

```
## Design matrix: 27 features
```

Outer 5-Fold CV with Inner Lambda Selection

We run a nested cross-validation loop. The outer loop splits stays into 5 folds, and the inner loop (inside `cv.glmnet`) selects the optimal regularization parameter lambda on each training fold. Out-of-sample predictions are collected for honest evaluation.

```
folds <- sample(rep(1:5, length.out = nrow(X)))
oos_en <- numeric(nrow(X)) # elastic-net OOS risk
ALPHA <- 0.5 # elastic net (LASSO-ish)
for (k in 1:5) {
```

```

tr <- folds != k
cvfit <- cv.glmnet(X[tr, ], y[tr], family = "binomial",
                  alpha = ALPHA, nfolds = 5, standardize = TRUE)
oos_en[!tr] <- as.numeric(predict(cvfit, X[!tr, ],
                                s = "lambda.min", type = "response"))
}

```

Reconstruct Baseline Models

We re-fit the baseline GLMs from the earlier script (09) on the same outer folds so that a fair DeLong test comparison can be made. Both the static-only and trajectory-augmented baselines are reconstructed.

```

# Reconstruct the earlier (script 09) baseline models on the SAME outer
# folds (for DeLong). Fit on the imputed matrix `df` so all three models use
# identical rows (the raw lm6 vitals carry a little missingness that would
# leave NA predictions).
gdat <- cbind(df, y = y)
cvauc_glm <- function(form, data) {
  p <- numeric(nrow(data))
  for (k in 1:5) { tr <- folds != k
    m <- glm(form, data[tr, ], family = binomial)
    p[!tr] <- predict(m, data[!tr, ], type = "response") }
  p
}
p_static <- cvauc_glm(y ~ age + sofa_total + qsofa + sirs +
                     hr + resp_rate + map + spo2 + temp_c, gdat)
p_traj <- cvauc_glm(y ~ age + sofa_total + qsofa + sirs +
                   hr + resp_rate + map + spo2 + temp_c +
                   hr_slope6 + resp_rate_slope6 + map_slope6 + spo2_slope6,
                   gdat)

```

Compute Metrics

We calculate AUROC, PR-AUC, and Brier score for each model, then run a DeLong test comparing the elastic-net model against the trajectory baseline. Results are saved to `tier1_metrics.csv`.

```

auc_of <- function(p) as.numeric(auc(roc(y, p, quiet = TRUE, direction = "<")))
prauc_of <- function(p) pr.curve(scores.class0 = p[y==1],
                                scores.class1 = p[y==0])$auc.integral
brier <- function(p) mean((p - y)^2)

roc_en <- roc(y, oos_en, quiet = TRUE, direction = "<")
roc_trj <- roc(y, p_traj, quiet = TRUE, direction = "<")
dl_vs_traj <- roc.test(roc_en, roc_trj, method = "delong")$p.value

metrics <- data.frame(
  model = c("Baseline static (glm)", "Baseline + trajectory slopes (glm, 0.755 AUROC)",
            "Tier1 full-feature elastic net"),
  auROC = c(auc_of(p_static), auc_of(p_traj), auc_of(oos_en)),
  prauc = c(prauc_of(p_static), prauc_of(p_traj), prauc_of(oos_en)),
  brier = c(brier(p_static), brier(p_traj), brier(oos_en)))
metrics$delong_p_vs_baseline_traj <- c(NA, NA, dl_vs_traj)
metrics[, -1] <- round(metrics[, -1], 4)
print(metrics, row.names = FALSE)

```

```
##                                model  auroc  prauc  brier
##                                Baseline static (glm) 0.7488 0.1735 0.0600
## Baseline + trajectory slopes (glm, 0.755 AUROC) 0.7546 0.1793 0.0598
##                                Tier1 full-feature elastic net 0.7752 0.2064 0.0587
## delong_p_vs_baseline_traj
##                                NA
##                                NA
##                                0
write.csv(metrics, file.path(OUT, "tier1_metrics.csv"), row.names = FALSE)
```

Final Model Coefficients

We fit the elastic-net model on all available data and extract the penalized coefficients and odds ratios. Only features with non-zero coefficients are retained and saved to `tier1_coefs.csv`.

```
cvfull <- cv.glmnet(X, y, family = "binomial", alpha = ALPHA,
                    nfolds = 5, standardize = TRUE)
co <- as.matrix(coef(cvfull, s = "lambda.min"))
coefs <- data.frame(term = rownames(co), coef = as.numeric(co))
coefs$odds_ratio <- exp(coefs$coef)
coefs <- coefs[coefs$coef != 0, ]
coefs[, c("coef", "odds_ratio")] <- round(coefs[, c("coef", "odds_ratio")], 4)
write.csv(coefs, file.path(OUT, "tier1_coefs.csv"), row.names = FALSE)
cat(sprintf("\nSelected features (non-zero at lambda.min): %d / %d\n",
            nrow(coefs) - 1, ncol(X)))
```

```
##
## Selected features (non-zero at lambda.min): 25 / 27
```

Export Out-of-Sample Predictions

The per-stay out-of-sample risk estimates are exported to `tier1_oos_pred.csv`. These feed into the Tier 4 alarm/utility analysis downstream.

```
write.csv(data.frame(stay_id = lm6$stay_id, onset_within_h = y,
                    risk_en = oos_en, risk_baseline_traj = p_traj),
          file.path(OUT, "tier1_oos_pred.csv"), row.names = FALSE)
```

Calibration Curve

We produce a decile-binned calibration plot comparing predicted risk against observed onset rate. This plot is saved to `figure/calibration_h6.png`.

```
png(file.path(FIG, "calibration_h6.png"), width = 1000, height = 1000, res = 200)
bins <- cut(oos_en, quantile(oos_en, seq(0, 1, 0.1)), include.lowest = TRUE)
cal <- aggregate(cbind(pred = oos_en, obs = y), list(bin = bins), mean)
plot(cal$pred, cal$obs, pch = 19, xlab = "Predicted risk",
     ylab = "Observed onset rate", main = "Calibration (out-of-sample, h6)",
     xlim = range(cal$pred, cal$obs), ylim = range(cal$pred, cal$obs))
abline(0, 1, lty = 2, col = "grey50"); lines(cal$pred, cal$obs, col = "steelblue")
dev.off()
```

```
## pdf
## 2
```

Nomogram

We refit an unpenalized logistic regression model (`lrm`) on the features selected by elastic net, then render a clinician-facing nomogram. The figure is saved to `figure/nomogram_h6.png`.

```
sel <- setdiff(coefs$term, "(Intercept)")
nomo_df <- df[, intersect(sel, names(df)), drop = FALSE]; nomo_df$y <- y
dd <- datadist(nomo_df); options(datadist = "dd")
frm <- as.formula(paste("y ~", paste(names(nomo_df)[names(nomo_df) != "y"],
                                   collapse = " + ")))
fit_lrm <- lrm(frm, data = nomo_df, maxit = 1000)
png(file.path(FIG, "nomogram_h6.png"), width = 1800, height = 1400, res = 180)
plot(nomogram(fit_lrm, fun = plogis, funlabel = "Risk of sepsis within 12h"),
     cex.axis = 0.6, cex.var = 0.7)
dev.off()
```

```
## pdf
## 2
```

Done

```
cat("\nTier 1 complete. Outputs in", OUT, "and", FIG, "\n")
```

```
##
## Tier 1 complete. Outputs in ../../processed_data and figure
```